Enter your choice 4

5.7 APPLICATIONS OF STACKS

There are a number of applications of stacks, some of them are discussed briefly in the preceding sections.

5.7.1 Stack Frames

Almost invariably, programs compiled from modern high level languages (even C) make use of a stack frame for the working memory of each procedure or function invocation. When any procedure or functions is called, a number of words — the stack frame — is pushed onto a program stack. When the procedure or function returns, this frame of data is popped off the stack.

As a function calls another function, first its arguments, then the return address and finally space for local variables is pushed onto the stack. Since each function runs in its own "environment" or **context**, it becomes possible for a function to call itself — a technique known as ***recursion***. This capability is extremely useful and extensively used — because many problems are elegantly specified or solved in a recursive way.

Program stack after executing a pair of mutually recursive functions :

```
Function f(int x, int y) {
int a ;
```

Scanned by TapScanner

```
if (term_cond) return ......;
a = ..... ;
return g(a) ;
function g(int z)
{
   int p, q ;
   p = ... ; q = .. ;
   return f(p, q) ;
}
```

Note how all of functions f and g's environment (their parameters and local variables) are found in the stack frame. When f is called a second time from g, a new frame for the second invocation of f is created.

The stack is a region of main memory within which programs temporarily store data as they execute. For example, when a program sends parameters to a function, the parameters are placed on the stack. When the function completes its execution these parameters are popped off from the stack. When a function calls other function the current contents (*i.e.*, variables) of the caller function are pushed onto the stack with the address of instruction just next to the call instruction, this is done so that after the execution of called function, the compiler can track back (remember) the path from where it is sent to the called function.

5.7.2 Reversing a String

As the characteristic of stack is reversing the order of execution. This fact can be exploited to reverse strings of line of characters. This can be simply thought of simply as pushing the individual characters, and when the complete line is pushed onto the stack, then individual characters of the line are popped off. Because the last inserted character pushed on stack would be the first character to be popped off, the line obviously be reversed.

5.7.3 Calculation of Postfix Expression

Another application of stack is calculation of postfix expression. To understand what is a postfix expression let us undergo a discussion. There are basically three types of notation for an expression (mathematical expression ; An expression is defined as a number of operands or data items combined using several operators.) :

- Infix notation
- Prefix notation
- Postfix notation

Infix

The **infix** notation is what we come across in our general mathematics, where the operator is written in-between the operands. For example : The expression to add two numbers A and B is written in infix notation as :

```
A + B
```

Note that the operator '+' is written in-between the operands A and B. The reason why this notation is called *infix*, is the place of operator in the expression.

Scanned by TapScanner

Prefix

The **prefix** notation a notation in which the operator is written before the operands, it is also called **polish** notation in the honor of the mathematician Jan Lukasiewicz who developed this notation. The same expression when written in prefix notation looks like :

```
+ AB
```

As the operator '+' is written before the operands A and B, this notation is called prefix (pre means before).

Postfix

In the **postfix** notation the operators are written after the operands, so it is called the postfix notation (post means after), it is also known as **suffix** notation or **reverse polish notation**. The above expression if written in postfix expression looks like follows :

```
AB +
```

The prefix and postfix notations are not really as efficient to use as they might look at first. For example, a C function to return the sum of two variables A and B (passed as argument) is called or invoked by the instruction :

```
add(A, B)
```


Note the operator ***add*** (name of the function) precedes the operands A and B. Because the postfix notation is a type of notation which is most suitable for a computer to calculate any expression (due to its reversing characteristic), and is the universally accepted notation for designing arithmetic and logical (ALU) unit of the CPU (processor). Therefore it is necessary for us to study the postfix notation. Moreover the postfix notation is the way computer looks towards any arithmetic expression, any expression entered into the computer is first converted into postfix notation, stored in stack and then calculated. In the preceding sections we will study the conversion of an expression from one notation to other.

Advantage of using postfix notation

Although human beings are quite used to work with mathematical expressions in *infix* notation, which is rather complex, as using this notation one has to remember a set of non-trivial rules. That set of rules must be applied to expressions in order to determine the final value. These rules include precedence, BODMAS, and associativity.

Using infix notation one cannot tell the order in which operators should be applied by only looking at expression. Whenever an infix expression consists of more than one operators, the precedence rules (BODMAS) should be applied to decide that which operator (and operands associated with that operator) are evaluated first. As compared to postfix notation, which is much more easy to work with or evaluated first. As compared to postfix notation, which is much more easy to work with or evaluate. In a postfix expression operands appear before the operators, there is no need for operator precedence and other rules. As soon as an operator appears in the postfix expression during scanning of postfix expression the topmost operands are popped off and are calculated applying the encountered operator. Place the result back onto the stack, doing so the stack will contain finally a single value at the end of process.

Scanned by TapScanner

5.7.3A Notation Conversions

Let an expression A + B * C is given in, which is in *infix* notation. To calculate this expression for values 4, 3, 7 for A, B, C respectively we must follow certain rule (called BODMAS in general mathematics) in order to have right result. *For example* :

A + B * C = 4 + 3 * 7 = 7 * 7 = 49

Is this the right result ? No, this is because the multiplication is to be done before addition, because it has higher precedence over addition. This means that for an expression to be calculated we must have the knowledge of precedence of operators. The error in the above calculation occurs, as there are no braces to define the precedence of operators. Thus expression A + B * C is interpreted as, A + (B * C). This is an alternative for us to convey the computer that multiplication has higher precedence over addition, as there is no other way to specify this.

Operator precedence

Exponential operator	^	Highest precedence
Multiplication/Division	*, /	Next precedence
Addition/Subtraction	+, –	Least precedence

Converting infix expression to postfix form

A + B * C	Infix Form
A + (B * C)	Parenthesized expression
A + (BC*)	Convert the multiplication
A (BC*) +	Convert the addition
ABC* +	Postfix form

The rules to be remembered during *infix* to *postfix* conversion are :

(*i*) Parenthesize the expression starting from left to right.

(*ii*) During parenthesizing the expression, the operands associated with operator having higher precedence are first parenthesized. For example in above expression B * C is parenthesized first before A + B.

(*iii*) The sub-expression (part of expression) which has been converted into postfix is to be treated as single operand.

(*iv*) Once the expression is converted to postfix form remove the parenthesis.

Let us consider some solved problems for converting infix expression to postfix, we will follow the above said four rules.

EXAMPLE 5.3

Give postfix form for $A + [\ (B + C) + (D + E) * F\ ] / G$

SOLUTION. Evaluation order is

A + [{(BC +) + (DE+) * F } / G]

Scanned by TapScanner

A + [{(BC +) + (DE + F *)} / G]
A + [{BC + (DE + F * +} / G]
A + (BC + DE + F * + G/)
ABC + DE + F * + G/ + **Postfix Form**

EXAMPLE 5.4

Give postfix form for $A + B - C$.

SOLUTION. (A + B) – C
(AB +) – C

Now as said-after converting a part of expression to postfix, treat it as single unit, therefore (AB+) now can be treated as T.

T – C
TC –

Now as the final conversion is accomplished replace the value of T.

AB + C – **Postfix Form**

EXAMPLE 5.5

*Convert the expression A * B + C/D to postfix form.*

SOLUTION. In this particular example the * operator is encountered during left to right evaluation, which shares an equal precedence to operator. When this happens (two operators with same precedence comes in same expression), the operator that is encountered first while evaluating from left to right is first parenthesized. Hence the above expression will be interpreted as :

(A * B) + C/D
(AB *) + C/D
(T) + C/D
(T) + (C/D)
(T) + (CD/)
T + S S = CD/
TS +
AB * CD/+. **Postfix expression.**

EXAMPLE 5.6

*Give positive form for A * B + C.*

SOLUTION. (A * B + C)
(AB *) + C
(T) + C
TC +
AB * C+. **Postfix expression.**

Scanned by TapScanner

EXAMPLE 5.7

Give postfix form for $A + B/C - D$.

SOLUTION.

A + (B/C) – D
A + (BC/) – D
A + T – D
(A + T) – D
(AT+) – D
S – D where $S = (AT +)$
SD –
AT + D –
ABC / + D –. **Postfix expression.**

EXAMPLE 5.8

Convert the expression $(A + B) / (C - D)$ *to postfix form.*

SOLUTION. In this expression the brackets are already specified. Therefore the conversion looks like :

(AB+) / (CD–)
T/S where T = (AB+) and S = (CD–)
TS/
AB + CD-/. **Postfix expression.**

EXAMPLE 5.9

Give postfix form for $(A + B) * C/D$.

SOLUTION.

(AB+) * C/D
T * C/D where T = (AB+)
(T * C)/D
(TC *)/D
S/D
SD/
TC * D/
AB + C * D/. **Postfix expression.**

EXAMPLE 5.10

Give postfix form for $(A + B) * C/D + E \wedge F/G$.

SOLUTION.

(AB+) * C/D + E ^ F/G
T * C/D + (E ^ F) / G (^ has the highest priority)
T * C/D + (EF ^) / G

STACKS

Scanned by TapScanner

T * C/D + S/G where S = (EF ^)
(T * C)/D + S/G
(TC *)/D + S/G
Q/D + S/G where Q = (TC *)
(Q/D) + S/G
(QD/) + S/G
P + S/G where P = (QD/)
P + (S/G)
P + (SG/)
P + O where O = (SG/)
PO+

Now we will expand the expression PO+

PO+
QD/O+
TC * D/O+
AB + C * D/SG/+
AB + C * D/EF ^ G/+. **Postfix expression.**

EXAMPLE 5.11

*A + [(B + C) + (D + E) * F]/g. Give postfix form.*

SOLUTION. A + [(BC+) + (DE+) * F]/G
A + [T + S * F]/G where T = (BC+) and S = (DE+)
A + [T + (S * F)]/G
A + [T + (SF*)]/G
A + [T + Q]/G where Q = (SF*)
A + (TQ+)/G
A + P/G where P = (TQ+)
A + (PG/)
A + N where N = (PG/)
AN+

Expanding the expression AN+

APG/+
ATQ + G/+
ABC + Q + G/+
ABC + SF * + G/+
ABC + DE + F * G/+ **Postfix expression.**

Scanned by TapScanner

EXAMPLE 5.12

*A + (B * C – (D / E ^ F) * G) * H. Give postfix form.*

SOLUTION.

A + (B * C – (D/(EF ^) * G)) * H (^ has the highest priority)
A + (B * C – (D/T) * G)) * H where T = EF^
A + (B * C – ((D/T) * G)) * H
A + (B * C – ((DT/) * G)) * H
A + (B * C – (S * G)) * H where S = DT/
A + (B * C – (SG *)) * H
A + (B * C – Q) * H
A + ((B * C) - Q) * H where Q = SG *
A + ((BC *) – Q) * H
A + (P – Q) * H where P = (BC *)
A + (PQ-) * H
A + O * H where O = (PQ –)
A + (O * H)
A + (OH *)
A + N where N = (OH *)
AN+

Expanding the expression AN+

AOH * +
APQ – H * +
ABC * Q – H * +
ABC * SG * – H *+
ABC * DT/G * – H * +
ABC * DEF ^ /G * – H * +. **Postfix expression.**

EXAMPLE 5.13

*A – B/(C * D ^ E). Give postfix form.*

SOLUTION.

A – B/(C * (DE ^))
A – B/(C * T) where T = (DE^)
A – B/(CT *)
A – B/S where S = (CT *)
A – (B/S)
A – (BS/)
A – Q where Q = (BS/)
AQ –

Scanned by TapScanner

Expanding the expression AQ –

AQ –
ABS/-
ABCT */–
ABCDE ^*/– **Postfix expression.**

Converting infix expression to prefix form :

A + B * C	Infix form
A + (B * C)	Parenthesize Expression
A + (* BC)	Convert multiplication
+ A (*BC)	Convert addition
+ A * BC	Prefix form

The rules for *infix* to *prefix* conversion are same simple and similar to previous ones. Let us consider some solved problems for converting infix to prefix.

EXAMPLE 5.14

*Give prefix form for A * B + C.*

SOLUTION. (A * B) + C
(* AB) + C
T + C where T = (* AB)
+TC

expanding expression +TC

+ * ABC. **Prefix expression.**

EXAMPLE 5.15

Give prefix form for A/B ^C + D.

SOLUTION. A/(B ^ C) + D (^has highest priority)
A/(^BC) + D
A/T + D where T = (^BC)
(A/T) + D
(/AT) + D
S + D where S = (/AT)
+SD

Expanding expression +SD

+/ATD
+/A ^ BCD. **Prefix expression.**

EXAMPLE 5.16

*Give prefix form for (A – B/C) * (D * E – F).*

SOLUTION. (A – (B/C)) * ((D * E) – F)

Scanned by TapScanner

(A – (/BC)) * ((*DE)) – F)
(A – T) * (S – F)
(–AT) * (–SF) where (T = /BC) and S = (*DE)
Q * P
* QP where Q = (–AT)

Expanding expression *QP

*AT – SF
* – A/BC – SF
* – A/BC –* DEF. **Prefix expression.**

EXAMPLE 5.17

*Give prefix form for (A * B + (C/D)) – F*

SOLUTION.

(A * B + (/CD)) – F
(A * B + T) – F
(A * B + T) – F
((A * B) + T) – F where (T = /CD)
((*AB) + T) – F
(S + T) – F
(+ST) – F where S = (* AB)
Q – F
–QF where Q = (+ST)

Expanding expression –QF

– +STF
– + *ABTF
– + *AB/CDF. **Prefix expression.**

EXAMPLE 5.18

A/B ^ C – D. Give prefix expression.

SOLUTION.

A/(B ^ C) – D
A/(^BC) – D
A/T – D where T = (^BC)
(A/T) – D
(/AT) – D
S – D where S = (/AT)
– SD

Expanding expression –SD

–/ATD
–/A ^ BCD. **Prefix expression.**

STACKS

Scanned by TapScanner

↑ → 1
*/ → 2
+ - → 3.

(1) NO 2 op of same priority can stay together

(2) new op : lower than old top
the old higher op will poped

** G) * H into postfix form showing stack status after every step in*

H into postfix form.

Stack	Expression
	A
	A

Scanned by TapScanner

(A + B / C * (D + E) - F)

Symbol	Stack	postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
/	(+ /	AB

Here div. is higher than + so / can stay

C	(+ /	ABC

Here * and / are same priority No two same priority stay together so the operator which is in stack before this will pop and * will push.

*	(+ *	ABC/
(	(+ * (	ABC/D
D	(+ * (	ABC/D
+	(+ * (+	ABC/DE
E	(+ * (+	ABC/DE
)	(+ * (+)	ABC/DE+

(+ *) pop

Scanned by TapScanner

) (+ * ABC/DE+

* There are * before In Stack but now * is lower ~~higher~~ it can not be done so the ~~op~~ operator which is in before (*) this is pop.

- (+ - ABC/DE+*

But here also - is same to + so ~~is~~ + is pop

F (- ABC/DE+*+F

) (-) ABC/DE+*+F-

Scanned by TapScanner

Infix to prefix

Rule 1: Two operator of same priority can stay together

Rule 2: New operator with higher priority than older operator can stay together

Rule 3: New operator with lower priority than older operator can not stay together.

Q: $(A + B / C * (D+E) - F)$.

Rev= $(F - (E+D) * C / B + A)$

Symbol	Stack	prefix
(	(	
F	(	F
-	(-	F
(	(-(	F
E	(-(	FE
+	(-(+	FE
D	(-(+	FED
)	(-(+) ↳pop	FED+
*	(-*	FED+

Note Here * is greater the - so they can stay together.

C (-* FED+C.

Scanned by TapScanner

Symbol	Stack	prefix.
/	(-*/	
	Here * and / are same priority So they can stay together	
E	(-*/	FE
D	(-*/	FED
+	(-*/+ → pop	FED+
	Here new operator (+) is lower priority than old (/) so old operator will pop. So we can get ~~but~~	
C	(-*+ → pop	FED+C
	but (+) is also lower than (*) So again * is pop because * is older. [(-+].	
B	(-+	FED+CB
/	(-+)/ → pop	FED+CB/
	Here + is popped 1st the -	
A		FDE+CB/*A
		FDE+CB/*A+- Ans

Scanned by TapScanner